

The background image is a composite of several elements. At the bottom, a person's hand is seen typing on a laptop keyboard. Above the laptop, a robotic hand with a glowing blue light on its index finger is positioned as if interacting with a transparent digital screen. This screen displays various data visualizations, including bar charts, line graphs, and pie charts. The entire scene is overlaid with a network of glowing blue lines and nodes, suggesting a complex data system or artificial intelligence interface. The overall color palette is dominated by light blues and greys, with the glowing elements providing a focal point.

CREATING AND MANAGING TABLES

Data Definition Language (DDL)

This language is used to define the tables. It sets up, changes, and removes data structures from the tables. It uses 5 commands, which are:

CREATE

ALTER

DROP

RENAME

TRUNCATE

Naming Rules

Table names and column names:

Must begin with a letter

Must be 1 to 30 characters long

Must contain only A–Z, a–z, 0–9, _, \$, and #

Must not duplicate the name of another object owned by the same user

Must not be an Oracle Server reserved word

Tables in the Database

User tables:

- **Are a collection of tables created and maintained by the user**
- **Contain user information**

Data dictionary:

- **Is a collection of tables created and maintained by the SQL Server**
- **Contain database information**

Creating Tables

The **CREATE TABLE** statement is used to create a new table in a database.

Syntax

```
CREATE TABLE table_name (  
    column1 datatype,  
    column2 datatype,  
    column3 datatype,  
    ....  
);
```

Creating Tables

The following example creates a table called "Persons" that contains five columns: PersonID, LastName, FirstName, Address, and City:

```
CREATE TABLE Persons (  
    PersonID int,  
    LastName varchar(255),  
    FirstName varchar(255),  
    Address varchar(255),  
    City varchar(255)  
);
```

The empty "Persons" table will now look like this:

PersonID	LastName	FirstName	Address	City

Creating Tables

Create the table

```
CREATE TABLE dept
      (deptno NUMBER(2) ,
       dname  VARCHAR2(14) ,
       loc    VARCHAR2(13)) ;
Table created.
```

Confirm creation of the table

```
DESCRIBE dept
```

Name	Null?	Type
DEPTNO		NUMBER(2)
DNAME		VARCHAR2(14)
LOC		VARCHAR2(13)

Data Types

Data Type	Description
VARCHAR2 (<i>size</i>)	Variable-length character data
CHAR[(<i>size</i>)]	Fixed-length character data
NUMBER[(<i>p</i> , <i>s</i>)]	Variable-length numeric data
DATE	Date and time values
LONG	Variable-length character data up to 2 gigabytes
CLOB	Character data up to 4 gigabytes
RAW and LONG RAW	Raw binary data
BLOB	Binary data up to 4 gigabytes
BFILE	Binary data stored in an external file; up to 4 gigabytes
ROWID	Hexadecimal string representing the unique address of a row in its table

Create Table Using Another Table

A copy of an existing table can also be created using CREATE TABLE.

The new table gets the same column definitions. All columns or specific columns can be selected.

If you create a new table using an existing table, the new table will be filled with the existing values from the old table.

Create Table Using Another Table

```
CREATE TABLE new_table_name AS  
    SELECT column1, column2,...  
    FROM existing_table_name  
    WHERE ....;
```

Example

```
CREATE TABLE TestTable AS  
SELECT customername, contactname  
FROM customers;
```

The ALTER TABLE Statement

Use the ALTER TABLE statement to:

Add a new column

Modify an existing column

Drop a column

The ALTER TABLE Statement

Use the ALTER TABLE statement to add, modify or drop columns.

```
ALTER TABLE table
ADD          (column datatype [DEFAULT expr]
             [, column datatype]...);
```

```
ALTER TABLE table
MODIFY       (column datatype [DEFAULT expr]
             [, column datatype]...);
```

```
ALTER TABLE table
DROP        (column);
```

Adding a Column

DEPT80

EMPLOYEE_ID	LAST_NAME	ANNSAL	HIRE_DATE
149	Zlotkey	126000	29-JAN-00
174	Abel	132000	11-MAY-98
175	Taylor	103200	24-MAR-98

New column

JOB_ID

Add a new column to the DEPT80 table.

DEPT80

EMPLOYEE_ID	LAST_NAME	ANNSAL	HIRE_DATE	JOB_ID
149	Zlotkey	126000	29-JAN-00	
174	Abel	132000	11-MAY-98	
175	Taylor	103200	24-MAR-98	

Adding a Column

Use the **ADD** clause to add columns

```
ALTER TABLE dept80  
ADD          (job_id VARCHAR2(9));  
Table altered.
```

The new column becomes the last column

EMPLOYEE_ID	LAST_NAME	ANNSAL	HIRE_DATE	JOB_ID
149	Zlotkey	126000	29-JAN-00	
174	Abel	132000	11-MAY-96	
176	Taylor	103200	24-MAR-98	

Modifying a Column

You can change a column's data type, size.

```
ALTER TABLE dept80  
MODIFY      (last_name VARCHAR2(30));  
Table altered.
```

Dropping a Column

Use the DROP COLUMN clause to drop columns you no longer need from the table.

```
ALTER TABLE dept80  
DROP COLUMN job_id;  
Table altered.
```

Dropping a Table

All data in the table is deleted.

When you drop a table, the database loses all the data in the table

```
DROP TABLE dept80;  
Table dropped.
```

You *cannot* roll back the DROP TABLE statement

Changing the Name of an Object

To change the name of a table execute the RENAME statement

```
RENAME dept TO detail_dept;  
Table renamed.
```

You must be the owner of the object

Truncating a Table

The TRUNCATE TABLE statement:

- Removes all rows from a table
- Releases the storage space used by that table

You cannot roll back row removal when using TRUNCATE

Alternatively, you can remove rows by using the DELETE statement

Q/A